

Anatomy of a Shell

Every operating system has a shell, and to interact with it, we must use an application known as a **terminal emulator**. Here are some of the most common terminal emulators:

Terminal Emulator	Operating System
Windows Terminal	Windows
cmdr	Windows
PuTTY	Windows
kitty	Windows, Linux and MacOS
Alacritty	Windows, Linux and MacOS
xterm	Linux
GNOME Terminal	Linux
MATE Terminal	Linux
Konsole	Linux
Terminal	MacOS
iTerm2	MacOS

This list is by no means every terminal emulator available, but it does include some noteworthy ones. Also, because many of these tools are open-source, we can install them on different operating systems in ways that may differ from the developers' original intentions. However, that is a project beyond the scope of this module. Selecting the proper terminal emulator for the job is primarily a personal and stylistic preference based on our workflows that develop as we get familiar with our OS of choice. So don't let anyone make you feel bad for selecting one option over the other. The terminal emulator we interact with on targets will essentially be dependant on what exists on the system natively.

Command Language Interpreters

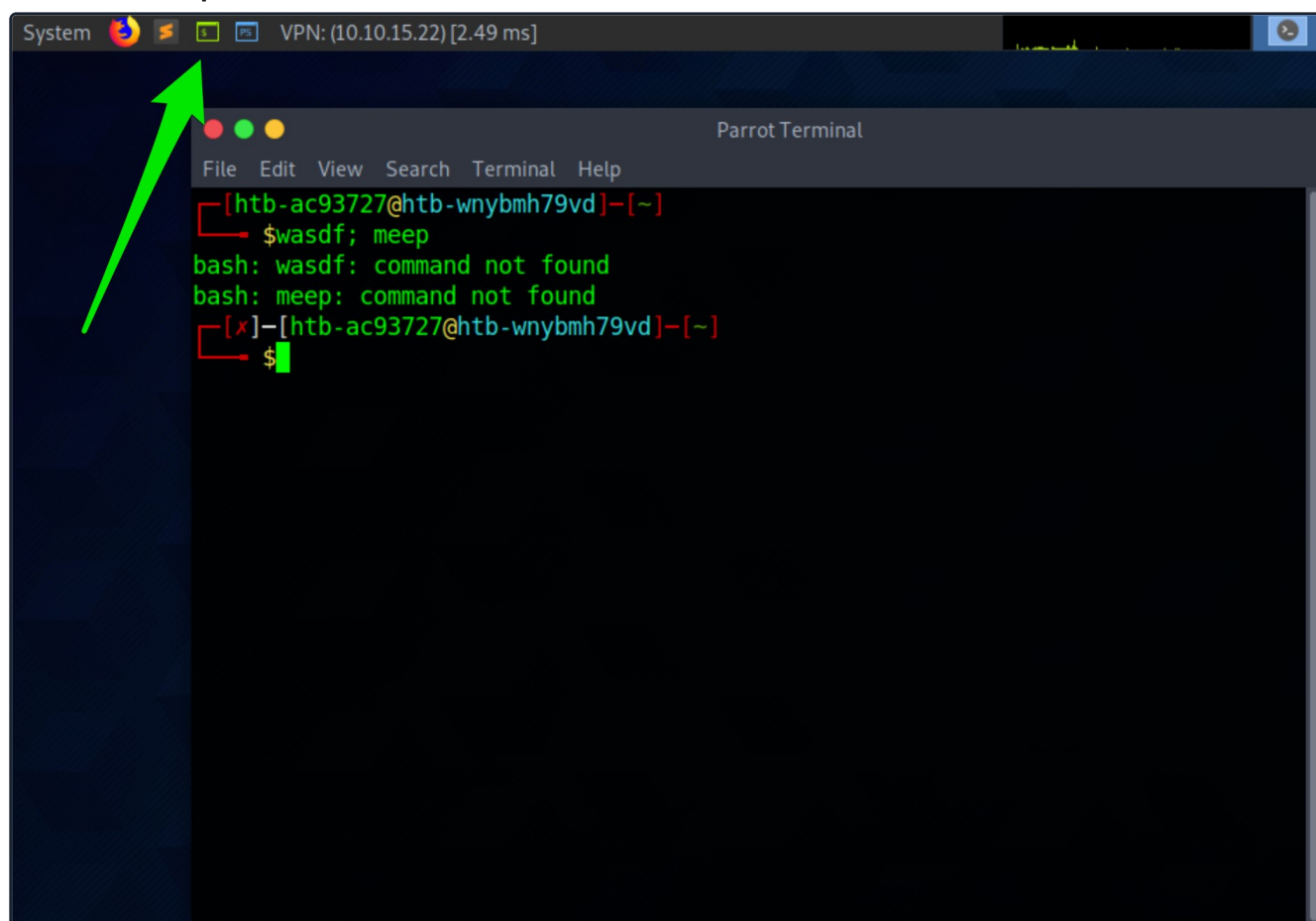
Much like a human language interpreter will translate spoken or sign language in real-time, a **command language interpreter** is a program working to interpret the instructions provided by the user and issue the tasks to the operating system for processing. So when we discuss command-line interfaces, we know it is a combination of the operating system, terminal emulator application, and the command language interpreter. Many different command language interpreters can be used, some of which are also called **shell scripting languages** or **Command and Scripting interpreters** as defined in the [Execution techniques](#) of the [MITRE ATT&CK Matrix](#). We do not need to be software developers to understand these concepts, but the more we know, the more success we can have when attempting to exploit vulnerable systems to gain a shell session.

Understanding the command language interpreter in use on any given system will also give us an idea of what commands & scripts we should use. Lets get hands-on with some of these concepts.

Hands-on with Terminal Emulators and Shells

Let's use our **Parrot OS** Pwnbox to further explore the anatomy of a shell. Click the **green** square icon at the top of the screen to open the **MATE** terminal emulator and then type something random and hit enter.

Terminal Example



As soon as we selected the icon, it opened the MATE terminal emulator application, which has been pre-configured to use a command language interpreter. In this instance, we are "clued" to what language interpreter is in use by seeing the **\$** sign. This **\$** sign is used in Bash, Ksh, POSIX, and many other shell languages to mark the start of the **shell prompt** where the user can begin typing commands and other input. When we typed out our random text and hit enter, our command language interpreter was identified. That is Bash telling us that it did not recognize that command we typed. So here, we can see command language interpreters can have their own set of commands that they recognize. Another way we can identify the language interpreter is by viewing the processes running on the machine. In Linux, we can do this using the following command:

Shell Validation From 'ps'

Shell Validation From 'ps'				
dbcyph0n@htb[/htb]\$ ps				
PID	TTY	TIME	CMD	
4232	pts/1	00:00:00	bash	
11435	pts/1	00:00:00	ps	

We can also find out what shell language is in use by viewing the environment variables using the **env** command:

Shell Validation Using 'env'

Shell Validation Using 'env'	
dbcyph0n@htb[/htb]\$ env	

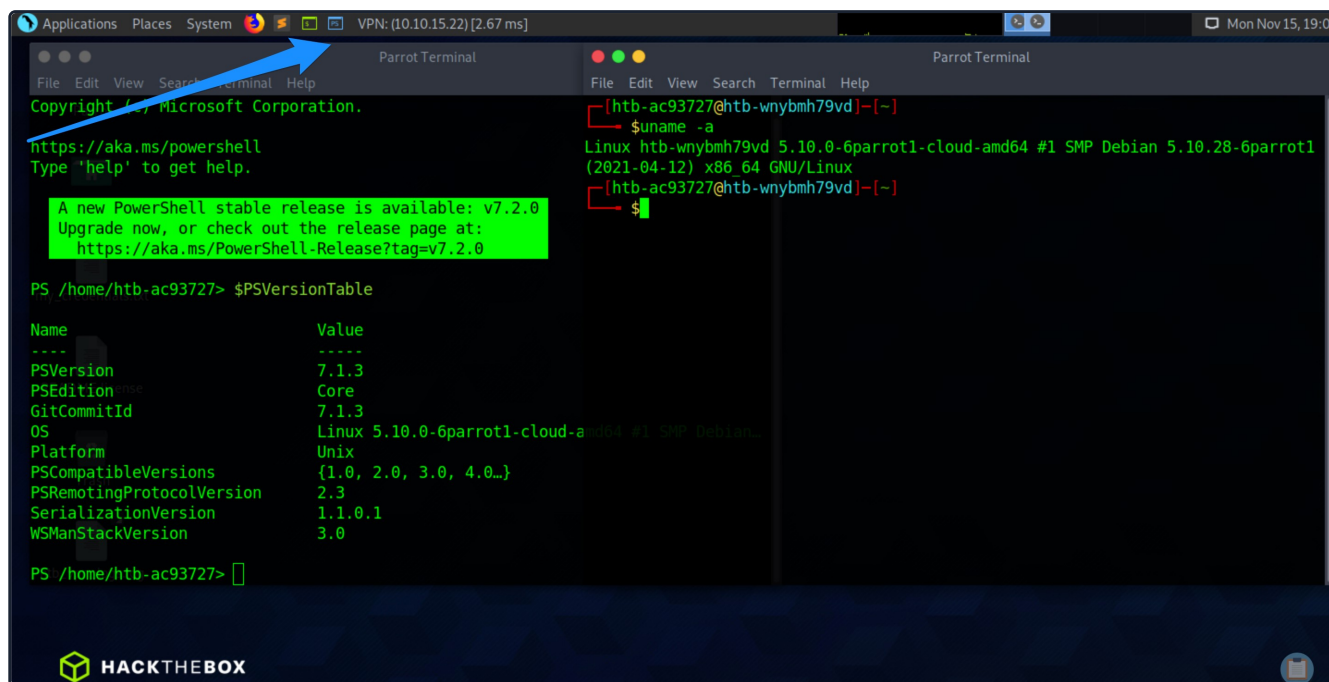
```

decrypt@htb:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
systemd:x:3:3:systemd:/var/lib/systemd/coredump:/usr/sbin/coredump
systemd-journal:x:4:65534:systemd-journal:/var/lib/systemd-journal:/usr/sbin/nologin
systemd-timesyncd:x:5:65534:systemd-timesyncd:/var/lib/systemd-timesyncd:/usr/sbin/nologin
ntp:x:7:7:ntp:/var/lib/ntp:/usr/sbin/nologin
sshd:x:65534:65534:sshd:/var/lib/ssh:/usr/sbin/nologin

```

Now let's select the blue square icon at the top of the screen in Pwnbox.

PowerShell vs. Bash



Selecting this icon also opens the MATE terminal application but uses a different command language interpreter this time around. Compare them as they are placed side-by-side.

- What differences can we identify?
- Why would we use one over the other on the same system?

There are countless differences and customizations we could discover. Try using some commands you know in both and make a mental note of the differences in output and which commands are recognized. One of the main points we can take away from this is a terminal emulator is not tied to one specific language. Actually, the shell language can be changed and customized to suit the sysadmin, developer, or pentester's personal preference, workflow, and technical needs.

Now for some challenge questions to test our understanding. All answers can be found utilizing Pwnbox.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

51ms ▾

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions



Cheat Sheet

Answer the question(s) below to complete this Section and earn cubes!

+ 0

Which two shell languages did we experiment with in this section? (Format: shellname&shellname)

Submit your answer here...



Submit



Hint

+ 0

In Pwnbox issue the \$PSVersionTable variable using PowerShell. Submit the edition of PowerShell that is running as the answer.

Submit your answer here...



Submit



Hint

← Previous

Next →



Cheat Sheet



Go to Questions

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

[Anatomy of a Shell](#)

 Bind Shells Reverse Shells

Payloads

Introduction to Payloads

 Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom

Windows Shells

 Infiltrating Windows

NIX Shells

 Infiltrating Unix/Linux

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

 Laudanum, One Webshell to Rule Them All Antak Webshell PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

 Start Instance / 1 spawns left